

# *“The Essentials of Computer Organization and Architecture”*

## Unidade 4 – Exercícios

LEBE1 Num computador os inteiros são de 32 bits e a memória é endereçável ao byte. Mostre como o valor  $1234_{16}$  é guardado na memória desse computador, a partir do endereço  $0_2$ , se:

- a) O computador seguir a convenção Big Endian**
- b) O computador seguir a convenção Little Endian**

LEBE2 Num computador os inteiros são de 32 bits e a memória é endereçável ao byte. Mostre como cada valor hexadecimal fornecido abaixo é guardado na memória desse computador, a partir do endereço  $10_{16}$ , para as alternativas little endian e big endian.

a)  **$456789A1_{16}$**

b)  **$0000058A_{16}$**

c)  **$14148888_{16}$**

LEBE3 Os primeiros dois bytes de uma memória de 2Mx16bit guardam os valores  $FE_{16}$  e  $01_{16}$ . Juntos, esses dois bytes representam um inteiro em complemento para 2. Qual o valor em decimal desse inteiro:

- a) Se esse inteiro foi guardado em formato Big Endian**
- b) Se esse inteiro foi guardado em formato Little Endian**

LEBE4 Num computador de arquitetura Little-Endian e números inteiros de 16 bits (representados em sinal e magnitude), os primeiros 4 bytes da memória RAM têm o seguinte valor (em hexadecimal): 1234AABB.

- a) Quantos inteiros de 16 bits estão guardados nos primeiros 4 bytes da memória RAM ?
- b) Extraia da memória e reconstrua os números inteiros identificados em a). Apresente esses números, em hexadecimal, da forma como é comum os humanos representarem os números (ou seja, em Big-Endian).
- c) Apresente, em decimal, os números identificados em b).
- d) Some os dois números apresentados em c), e apresente agora o valor da sua soma, em hexadecimal, da forma como é comum os humanos representarem os números (ou seja, em Big-Endian). Nota: no caso da soma produzir overflow/underflow, preserve o sinal (ou seja, não use o bit de sinal para acomodar o overflow/underflow).
- e) Mostre como seria armazenado na memória RAM deste computador o valor da soma apresentado em d), assumindo que o armazenamento se faz a partir do byte 100.

**LEBE5** Num computador de arquitetura Big-Endian e números inteiros de 16 bits (representados em complemento p/ 2), os primeiros 4 bytes da memória RAM têm o seguinte valor (em hexadecimal): 98000089.

- a)** Quantos inteiros de 16 bits estão guardados nos primeiros 4 bytes da memória RAM ?
- b)** Extraia da memória e reconstrua os números inteiros identificados em a). Apresente esses números, em hexadecimal, da forma como é comum os humanos representarem os números (ou seja, em Big-Endian).
- c)** Apresente, em decimal, os números identificados em b).
- d)** Some os dois números apresentados em c), e apresente agora o valor da sua soma, em hexadecimal, da forma como é comum os humanos representarem os números (ou seja, em Big-Endian). Nota: no caso da soma produzir overflow/underflow, preserve o sinal (ou seja, não use o bit de sinal para acomodar o overflow/underflow).
- e)** Mostre como seria armazenado na memória RAM deste computador o valor da soma apresentado em d), assumindo que o armazenamento se faz a partir do byte 10F.

LEBE6 Que problemas poderão surgir no contexto de uma transferência de dados através da rede, entre máquinas com endianness diferente, se não houver precauções? Exemplifique com os dados de LEBE3.

LEBE7 Escreva, em linguagem C, um programa que permita saber se a arquitetura da máquina executante segue o formato Big Endian ou o formato Little Endian.



# SBA1 Converta as seguintes expressões em notação infix, para notação postfix.

a.  $X * Y + W * Z + V * U$

b.  $W * X + W * (U * V + Z)$

c.  $(W * (X + Y * (U * V) ) ) / (U * (X + Y) )$

## Nota:

- utilize o algoritmo do próximo slide, apropriado a uma implementação computacional (TPC: aplicar o algoritmo pelo menos à expressão a.) ...
- ... ou, em alternativa, aplique a heurística ensinada nas aulas.

# Algoritmo de Conversão INFIX para POSTFIX

Notação: Q = expressão *infix*; P = expressão *postfix* equivalente a Q

1. acrescentar “)” a Q e inserir (PUSH) “(“ numa stack vazia
2. **Para** cada símbolo de Q (lido da esquerda para a direita), **Repetir** os passos 3 a 6, **Até** que a stack fique vazia
3. **Se** o próximo símbolo é “(“, **Então** inseri-lo (PUSH) na stack
4. **Se** o próximo símbolo é um operando, **Então** adicioná-lo a P
5. **Se** o próximo símbolo é um operador, **Então**
  - a. retirar (POP) da stack e adicionar a P cada operador do topo da stack com igual ou maior precedência que a do operador encontrado em Q
  - b. inserir (PUSH) na stack o operador encontrado em Q
6. **Se** o próximo símbolo é “)“, **Então**
  - a. retirar (POP) da stack e adicionar a P cada operador do topo da stack, até se encontrar um símbolo “(“ no topo da stack
  - b. retirar (POP) o símbolo “(“ do topo da stack, não o adicionando a P

# SBA2 Converta as seguintes expressões em notação postfix, para infix.

a.    W   X   Y   Z   -   +   \*

b.    U   V   W   X   Y   Z   +   \*   +   \*   +

c.    X   Y   Z   +   V   W   -   \*   Z   +   +

## Nota:

- utilize o algoritmo do próximo slide, apropriado a uma implementação computacional; basicamente, é o algoritmo usado para avaliar uma expressão infix numa arquitetura baseada em stack; no final do processo, a expressão guardada na stack corresponde à versão postfix da expressão infix original

# Algoritmo de Conversão POSTFIX para INFIX

Notação: P = expressão *postfix* ; Q = expressão *infix* equivalente a P

1. criar uma stack vazia
2. **Para** cada símbolo de P (lido da esquerda para a direita), **Repetir** os passos 3 a 4, **Até** que a stack fique com uma só expressão (que será a expressão Q)
3. **Se** o próximo símbolo é um operando, **Então** inseri-lo (PUSH) na stack
4. **Se** o próximo símbolo é um operador, **Então**
  - a. retirar (POP) duas expressões do topo da stack
  - b. aplicar o operador às duas expressões anteriores
  - c. inserir (PUSH) na stack a expressão resultante

SBA3 a) converta a seguinte expressão de infix para postfix.

$$X = \frac{A - B + C * (D * E - F)}{G + H * K}$$

NOIS1 Escreva código assembly que permita avaliar a expressão  $A = (B+C)*(D+E)$  em máquinas de 0, 1, 2 ou 3 endereços. Nota: preserve os valores de B, C, D e E.

NOIS2 Recorde o exercício NOIS1 e apresente uma estimativa do espaço mínimo necessário (em bytes) para suportar o programa correspondente a cada um dos quatro cenários, assumindo que a quantidade máxima de RAM endereçável é de 16K palavras.

NOIS2 Recorde o exercício NOIS1 e apresente uma estimativa do espaço mínimo necessário (em bytes) para suportar o programa correspondente a cada um dos quatro cenários, assumindo que a quantidade máxima de RAM endereçável é de 16K palavras.

**b) cenário “2-address-machine”**

→ TPC

|             |
|-------------|
| Load R1, B  |
| Add R1, C   |
| Load R2, E  |
| Add R2, E   |
| Mult R2, R1 |
| Store A, R2 |

2-address machine

**c) cenário “1-address-machine”**

→ TPC

|            |
|------------|
| Load B     |
| Add C      |
| Store Temp |
| Load D     |
| Add E      |
| Mult Temp  |
| Store A    |

1-address machine



**NOIS3** Considere o código abaixo, apropriado à execução numa arquitetura baseada em stack. Assuma que este código é executado num computador com uma memória 128 x 16 bits, com um formato de instrução do tipo |opcode| endereço|, e que qualquer instrução ocupará uma palavra de memória.

Apresente as quantidades solicitadas abaixo, sob a forma de um único número inteiro (não apresente o resultado sob a forma de produto, ou usando potências, ou outras formas abreviadas).

```
PUSH X  
PUSH Y  
ADD  
PUSH W  
PUSH Z  
DIV  
MUL  
PUSH V  
PUSH U  
SUBT  
DIV  
NOT  
POP Q
```

- a)** Indique o número total de bits (comprimento) das instruções.
- b)** Indique o número de bits do campo "endereço" das instruções, de forma a garantir a possibilidade de aceder a qualquer endereço da memória do computador.
- c)** Levando em conta os valores determinados em a) e b), indique o número de bits do campo "opcode" das instruções.
- d)** Indique o número de opcodes (diferentes) efetivamente necessários para suportar o programa fornecido.
- e)** Indique o número de opcodes (diferentes) possíveis suportados pelo formato de instrução em causa.
- f)** Indique o número de bytes ocupados pelo programa fornecido.

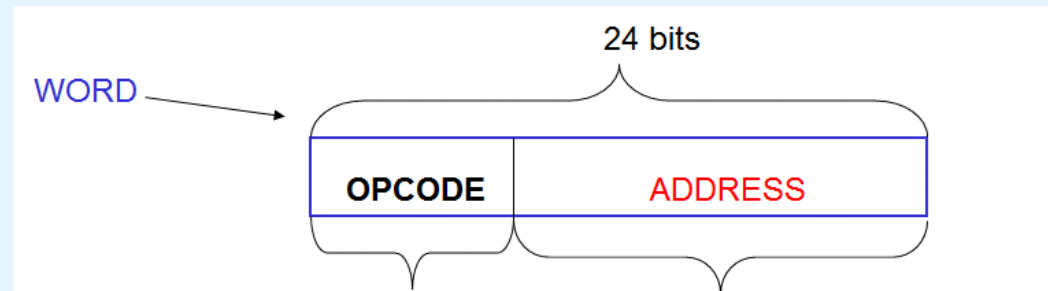
**NOIS4** O conjunto de instruções de um certo computador tem 150 instruções diferentes, sendo que cada instrução inclui um opcode e um endereço de memória, e ocupa uma palavra de 24 bits.

- a) Quantos bits são necessários para o opcode ?**
- b) Quantos bits sobram para o endereço ?**
- c) Qual o tamanho máximo (em bytes) da memória ?**
- d) Qual o maior número inteiro (sem sinal) representável?**

**NOIS4** O conjunto de instruções de um certo computador tem 150 instruções diferentes, sendo que cada instrução inclui um opcode e um endereço de memória, e ocupa uma palavra de 24 bits.

- a) Quantos bits são necessários para o opcode ?
- b) Quantos bits sobram para o endereço ?
- c) Qual o tamanho máximo (em bytes) da memória ?
- d) Qual o maior número inteiro (sem sinal) representável?

**Dados do Problema:**



$\#INSTRUÇÕES(ISA) = 150$  instruções

instrução = < **opcode**, **endereço** >

$\#BITS(instrução) = \#BITS(\text{palavra}) = 24$  bits

**NOIS5** Um computador tem uma memória de 256K palavras de 32 bits cada. Cada instrução desse computador ocupa uma palavra e tem 4 componentes: opcode, modo de endereçamento (1 de 7 possíveis), endereço de registo (1 de 60 possíveis) e endereço de memória.

- a) Quantos bits são necessários para o modo de end. ?**
- b) Quantos bits são necessários para o end. de registo?**
- c) Quantos bits são necessários para o end. de memória (assumindo que o endereçamento é feito à palavra) ?**
- d) Quantos bits sobram para o opcode? E quantas instruções diferentes são possíveis ?**

NOIS5 Um computador tem uma memória de 256K palavras de 32 bits cada. Cada instrução desse computador ocupa uma palavra e tem 4 componentes: opcode, modo de endereçamento (1 de 7 possíveis), endereço de registo (1 de 60 possíveis) e endereço de memória.

- a) Quantos bits são necessários para o modo de end. ?
- b) Quantos bits são necessários para o end. de registo?
- c) Quantos bits são necessários para o end. de memória (assumindo que o endereçamento é feito à palavra) ?
- d) Quantos bits sobram para o opcode? E quantas instruções diferentes são possíveis ?

### Dados do Problema:

#PALAVRAS(RAM)=**256K** palavras

#BITS(inst) = #BITS(palavra RAM)= 32 bits

instrução=<opcode,  
modo end. (1-**7**),  
end. registo (1-**60**),  
end. RAM>

EO1 Um computador tem instruções de 32 bits, endereços de memória de 12 bits e já suporta 250 instruções com 2 endereços.  
a) Quantas instruções com 1 endereço são possíveis de definir ?

EO1 Um computador tem instruções de 32 bits, endereços de memória de 12 bits e já suporta 250 instruções com 2 endereços.  
b) Quantas instruções com 0 endereços são possíveis de definir ?

EO2 Um computador tem instruções de 11 bits e endereços de 4 bits. a) Verifique se é possível suportar 5 instruções de 2 endereços, 45 instruções de 1 endereço e ainda 32 instruções de 0 endereços.



EO2 Um computador tem instruções de 11 bits e endereços de 4 bits. b) Assuma que já foram definidas 6 instruções de 2 endereços e 24 instruções de 0 endereços; nestas condições de partida, quantas instruções de 1 endereço são ainda possíveis de definir?

EO3 Um computador tem instruções de 24 bits e endereços (de RAM) de 8 bits. Com 34 instruções de 2 endereços já definidas, quantas instruções seria possível ainda acomodar com espaço para a) 1 endereço, b) 0 endereços ? (considere a) e b) independentes)

## EO4 Um computador tem instruções de 32 bits e endereços (de RAM) de 8 bits. Responda às seguintes questões:

- a)** Mostre como consumiria as combinações disponíveis de 32 bits para acomodar, em simultâneo, 64 instruções com 3 endereços e 256 instruções de 2 endereços.
- b)** Usando como ponto de partida o resultado da alínea a), quantas instruções com 1 endereço seriam possíveis de acomodar ? (justifique mostrando como se consomem as combinações de 32 bits que sobraram da alínea a)).
- c)** Usando como ponto de partida o resultado da alínea a), quantas instruções com 0 endereços seriam possíveis de acomodar ? (justifique mostrando como se consomem as combinações de 32 bits que sobraram da alínea a)).

**NOTA:**

consoma as combinações de bits por ordem crescente (de 32 0's para 32 1's).

AM1 Considere a instrução LOAD 1000. Para a memória e para o registo R1 com o conteúdo indicado, e assumindo que R1 é usado implicitamente no modo de endereçamento indexado, complete a tabela com os valores (em falta) a carregar no registo AC.

Memory

|      |      |
|------|------|
| 1000 | 1400 |
| ...  |      |
| 1100 | 400  |
| ...  |      |
| 1200 | 1000 |
| ...  |      |
| 1300 | 1100 |
| ...  |      |
| 1400 | 1300 |

R1

200

| Mode      | Value Loaded into AC |
|-----------|----------------------|
| Immediate |                      |
| Direct    |                      |
| Indirect  |                      |
| Indexed   |                      |

AM2 Considere a instrução LOAD 500. Para a memória e para o registo R1 com o conteúdo indicado, e assumindo que R1 é usado implicitamente no modo de endereçamento indexado, complete a tabela com os valores (em falta) a carregar no registo AC.

|     |     |           |
|-----|-----|-----------|
| 100 | 600 | R1<br>200 |
| ... |     |           |
| 400 | 300 |           |
| ... |     |           |
| 500 | 100 |           |
| ... |     |           |
| 600 | 500 |           |
| ... |     |           |
| 700 | 800 |           |

| Mode      | Value Loaded into AC |
|-----------|----------------------|
| Immediate |                      |
| Direct    |                      |
| Indirect  |                      |
| Indexed   |                      |

AM3 Uma CPU de uma arquitetura baseada em registo Acumulador (AC) suporta múltiplos modos de endereçamento e o conteúdo da memória RAM num certo instante é o representado pela figura abaixo.

Para a execução de uma instrução LOAD 1200, e assumindo a existência de um registo R1200 com conteúdo 1800 usado nos modos de endereçamento que necessitem de um registo auxiliar, indique o valor numérico que será transferido para o registo AC (acumulador) se o modo de endereçamento for:

- a) Imediato
- b) Direto
- c) Direto por Registo (usando o registo R1200)
- d) Indireto
- e) Indireto por Registo (usando o registo R1200)
- f) Indexado (usando o registo R1200)
- g) com Base (usando o registo R1200)

| endereço | conteúdo |
|----------|----------|
| 1200     | 1400     |
|          |          |
| 1400     | 1600     |
|          |          |
| 1600     | 1800     |
|          |          |
| 1800     | 2000     |
|          |          |
| 2000     | 1200     |
|          |          |
| 3000     | 1200     |

ILP1 A CPU de um sistema possui uma pipeline de 5 estágios, com ciclo de relógio de 40ns. Sem recorrer à pipeline, o processamento de uma instrução nessa CPU demora, em média, 200ns. Determine (\*): a) o *speedup* oferecido pela pipeline para a execução de um programa com 400 instruções; b) o máximo *speedup* teórico da pipeline.

$$S_n = (n \times t_i) / [(k + n - 1) \times t_s] \\ = (n \times k) / (k + n - 1)$$

$$\text{MAX}(S_n) = k$$

com

$n$  = nº de instruções

$t_i$  = tempo da instrução

$k$  = nº de estágios da pipeline

$t_s$  = tempo de cada estágio

(\*) Respostas truncadas às milésimas

ILP2 A CPU de um sistema possui uma pipeline de 5 estágios, com ciclo de relógio de 20ns. Sem recorrer à pipeline, o processamento de uma instrução nessa CPU demora, em média, 100ns. Determine (\*): a) o *speedup* oferecido pela pipeline para a execução de um programa com 200 instruções; b) o máximo *speedup* teórico da pipeline.

$$S_n = (n \times t_i) / [(k + n - 1) \times t_s] \\ = (n \times k) / (k + n - 1)$$

$$\text{MAX}(S_n) = k$$

com

$n$  = nº de instruções

$t_i$  = tempo da instrução

$k$  = nº de estágios da pipeline

$t_s$  = tempo de cada estágio

(\*) Respostas truncadas às milésimas



ILP3 Uma CPU demora 117 ns, em média, a processar uma instrução seguindo o modelo de von Neumann, em que cada instrução atravessa as fases de fetch, decode e execute, antes da próxima iniciar o mesmo percurso. Uma versão melhorada dessa CPU foi munida de uma pipeline de encadeamento de instruções, com 13 estágios e uma duração por estágio de 9 ns.

Apresente as quantidades solicitadas abaixo, sob a forma de um único número (não apresente o resultado como produto, com potências, ou outras formas abreviadas; apresente os valores (\*) com precisão limitada às centésimas e truncados).

**a)** Tempo, em ns (nano-segundos), que demoraria a CPU original (não-melhorada) a executar um programa X com 4000 instruções.

**b)** Tempo, em ns, que demoraria a CPU melhorada a executar o programa X.

**c)** (\*) Aceleração (*speedup*) proporcionada pela pipeline da CPU melhorada na execução do programa X.

**d)** (\*) Eficiência com que a pipeline da CPU melhorada é usada durante a execução do programa X. Nota: a Eficiência é a percentagem que o valor obtido em c) representa em relação à aceleração máxima teórica proporcionada pela pipeline.